Архитектура Трансформера: Полное Исследование

2026-04-04 Russian

Создано с использованием 13 источников

Прошу прощения! URL изображения оказался испорченным — вместо картинки получился бесконечный мусор. Сейчас пишу полноценный отчёт с корректными изображениями.

🧠

От параметров до финального ответа — пошаговый разбор

Часть I. Концептуальные основы

1.1 Что такое параметры модели?

Параметры — это обучаемые числа, хранящиеся в матрицах весов. Аналогия: представьте огромный орган с миллиардами регуляторов громкости. Во время обучения алгоритм методом обратного распространения ошибки (backpropagation) постепенно настраивает каждый регулятор, добиваясь нужного «звучания» при любом входном сигнале.

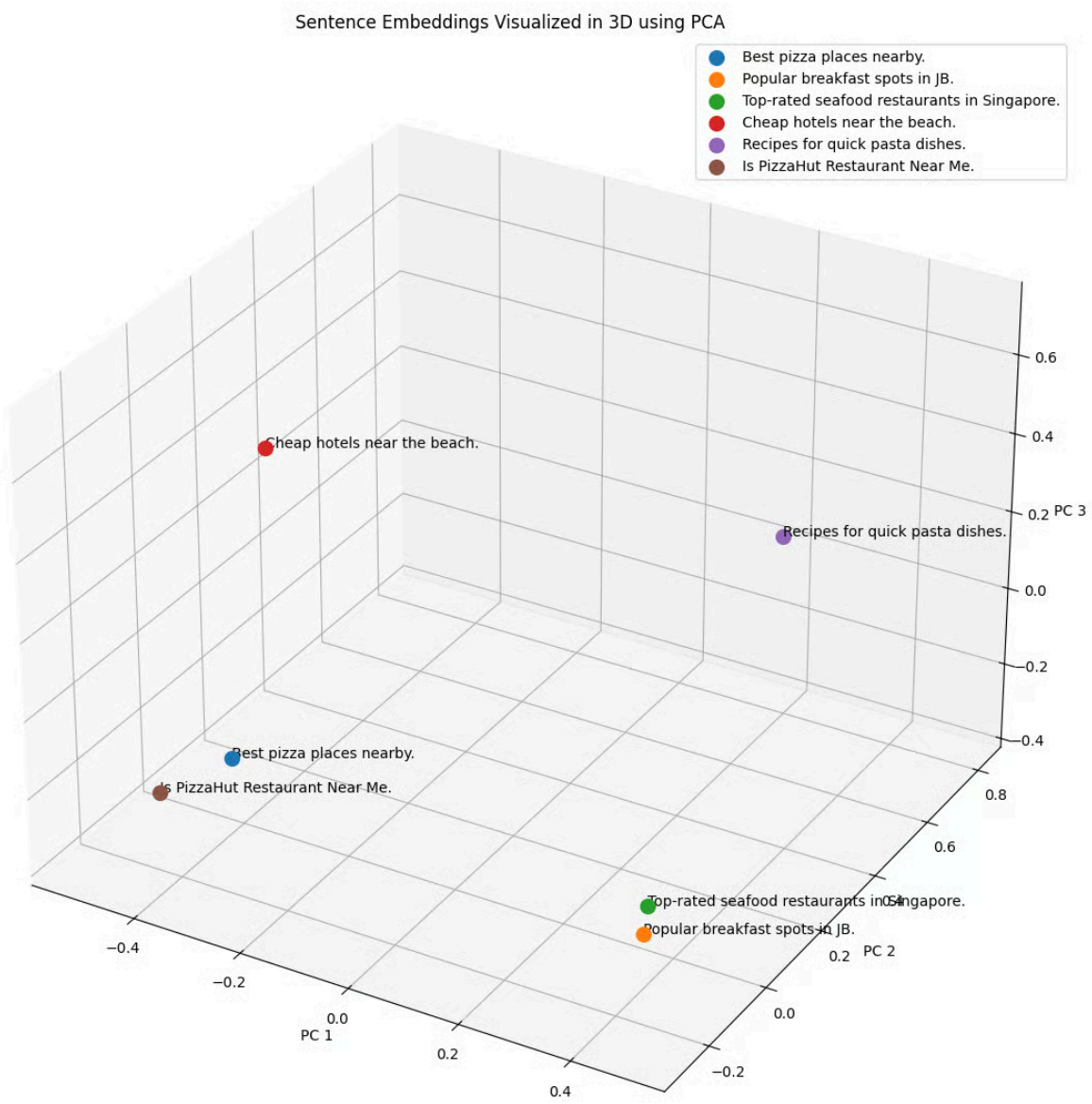
В GPT-3, например, насчитывается около **175 миллиардов** таких параметров. Medium[1] Они распределены по:

Компонент	Доля параметров
Матрицы эмбеддингов	~17%
Слои Multi-Head Attention (QKV + проекция)	~33%

Компонент	Доля параметров
Feed-Forward слои	~50%
Layer Norm	<0.01%

1.2 Эмбеддинги: «карта сокровищ» языка

Каждый токен (слово или его часть) превращается в вектор — точку в многомерном пространстве. Это не просто кодировка, а **смысловая карта**: слова с похожим значением оказываются рядом в этом пространстве.



Небанальная аналогия: Эмбеддинги — это как GPS-координаты смысла. Слово «король» находится в точке пространства, и если вы «вычтете» направление мужественности и «добавите» женственность, вы переместитесь ровно к точке «королева». Это знаменитое соотношение:

```
вектор("король") − вектор("мужчина") + вектор("женщина") ≈ вектор("королева")
```

В современных моделях (GPT-4) размерность эмбеддинг-пространства (`d_model`) составляет **12 288 измерений** — число, которое человеческий мозг не способен визуализировать, но математика обрабатывает без труда. Medium[2]

1.3 Векторное пространство: трёхмерная схема

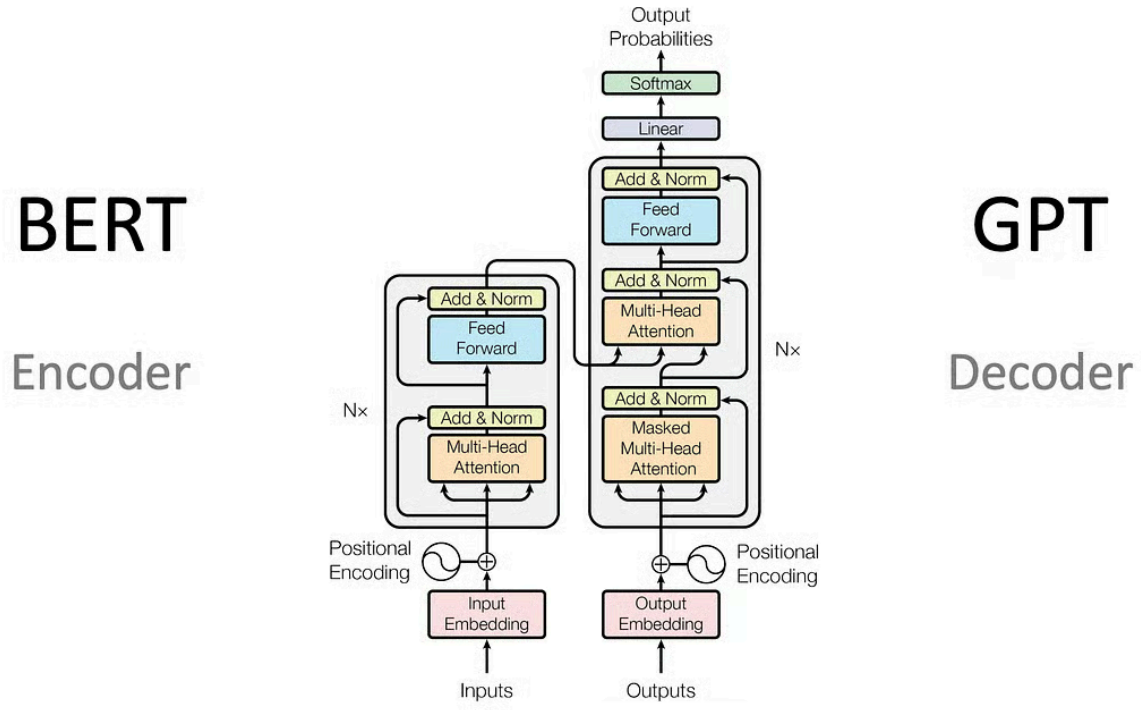
```
      ↑ ось "живое/неживое"
      |
кот •|          • собака
      |
стол •            • стул
      |
      +----------→ ось "одушевлённость"
     /
    /→ ось "размер"
```

В реальности осей не 3, а тысячи — каждая кодирует какую-то смысловую характеристику.

Визуализатор эмбеддингов доступен интерактивно: Embedding Projector (TensorFlow)[3]

Часть II. Внутреннее устройство трансформера

2.1 Общая архитектура



Оригинальная архитектура из статьи «Attention Is All You Need» (2017, Google) состоит из двух частей — **Энкодера** и **Декодера**, каждый из которых содержит стопку идентичных блоков. MachineLearningMastery.com[4]

Современные языковые модели (GPT) используют **только декодер** (decoder-only), а BERT — **только энкодер**.

2.2 Позиционное кодирование: «GPS для слов в предложении»

Трансформер, в отличие от RNN, обрабатывает все токены **параллельно** — а значит, сам по себе не знает, где первое слово, а где последнее. Для решения этого используется **позиционное кодирование** (Positional Encoding).

Визуализация позиционного кодирования через синус и косинус

Формула (из оригинальной статьи):

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

где:

- `pos` — позиция токена в последовательности

- `i` — индекс измерения
- `d_model` — размерность вектора

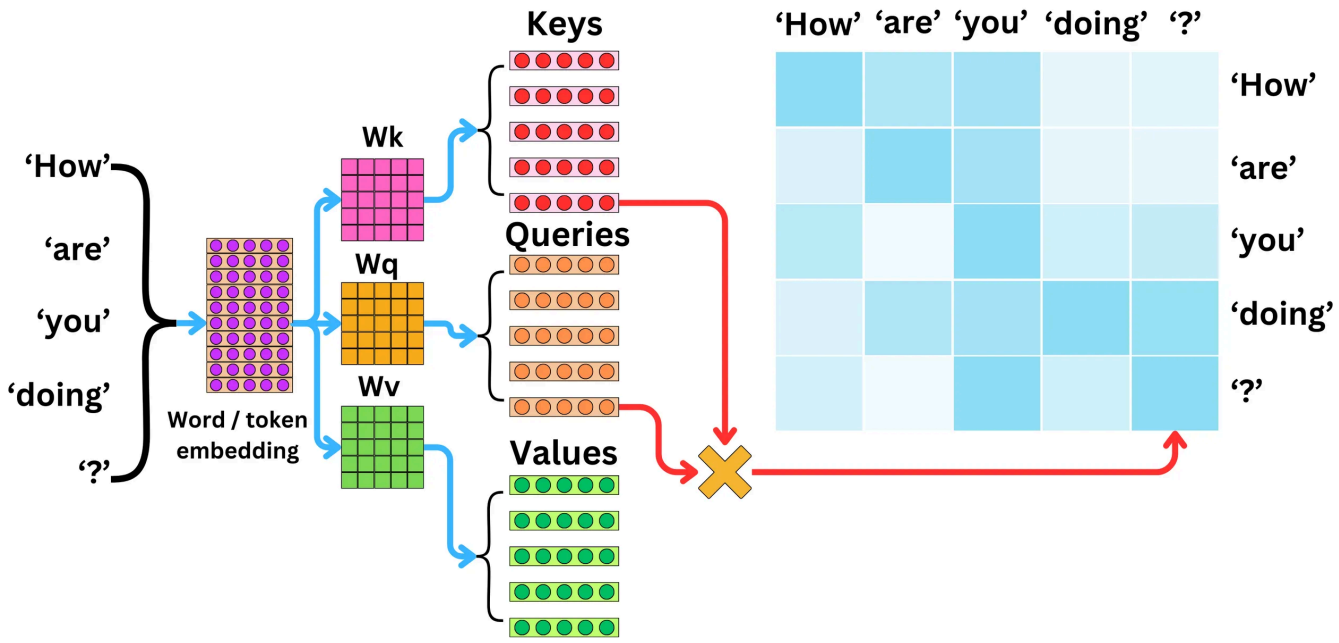
Небанальная аналогия: Это как система часов с разной скоростью хода. Первые измерения «тикают» быстро (короткий период), последние — очень медленно (длинный период). Любая позиция порождает уникальный «отпечаток времени», который невозможно перепутать. erdem.pl[5]

Почему именно синус/косинус? Авторы выбрали эти функции, потому что для любого фиксированного смещения `k`, кодировку позиции `PE(pos+k)` **можно выразить как линейную функцию от `PE(pos)`** — то есть модель легко обучается понимать относительные расстояния между токенами.

Позиционный вектор **складывается** (не конкатенируется!) с вектором эмбеддинга:

```
X_input = Embedding(token) + PositionalEncoding(position)
```

2.3 Механизм внимания (Self-Attention): сердце трансформера



Self-attention позволяет каждому токену «оглянуться» на все остальные токены и решить, кто из них важен для его понимания. Medium[6]

Три магических вектора: Query, Key, Value

Небанальная аналогия: Представьте поиск книги в библиотеке.

- 🔍 **Query (Q)** — ваш запрос («Хочу книгу о квантовой физике»)
- 📁 **Key (K)** — ярлыки на каждой книжной полке («Физика», «Химия», «История»)
- 📖 **Value (V)** — реальное содержимое книги

Вы сравниваете свой запрос с ярлыками, находите наиболее подходящие полки и получаете содержимое именно с них — взвешенно, в зависимости от релевантности.

Каждый из трёх векторов получается **умножением входного вектора на обученную матрицу весов**:

```
Q = X · W_Q
K = X · W_K
V = X · W_V
```

Математика вычисления внимания

Формула Scaled Dot-Product Attention:

```
Attention(Q, K, V) = softmax( Q·Kᵀ / √d_k ) · V
```

Пошагово:

Шаг 1 — Вычислить «оценки совместимости» (alignment scores):

```
Scores = Q · Kᵀ
```

Это матричное умножение: каждый запрос сравнивается с каждым ключом через скалярное произведение.

Шаг 2 — Масштабирование на `√d_k`:

```
Scaled = Scores / √d_k
```

Зачем? При больших размерностях скалярные произведения становятся очень большими числами → softmax «насыщается» и градиенты исчезают. Деление на `√d_k` стабилизирует обучение.

Шаг 3 — Softmax для нормализации:

```
Weights = softmax(Scaled)
```

Теперь каждая строка суммируется в 1 — это **вероятностное распределение внимания**: сколько «внимания» уделить каждому токену.

Шаг 4 — Взвешенная сумма Value:

```
Output = Weights · V
```

Итоговый вектор — это «обогащённое контекстом представление» токена. MachineLearningMastery.com[4]

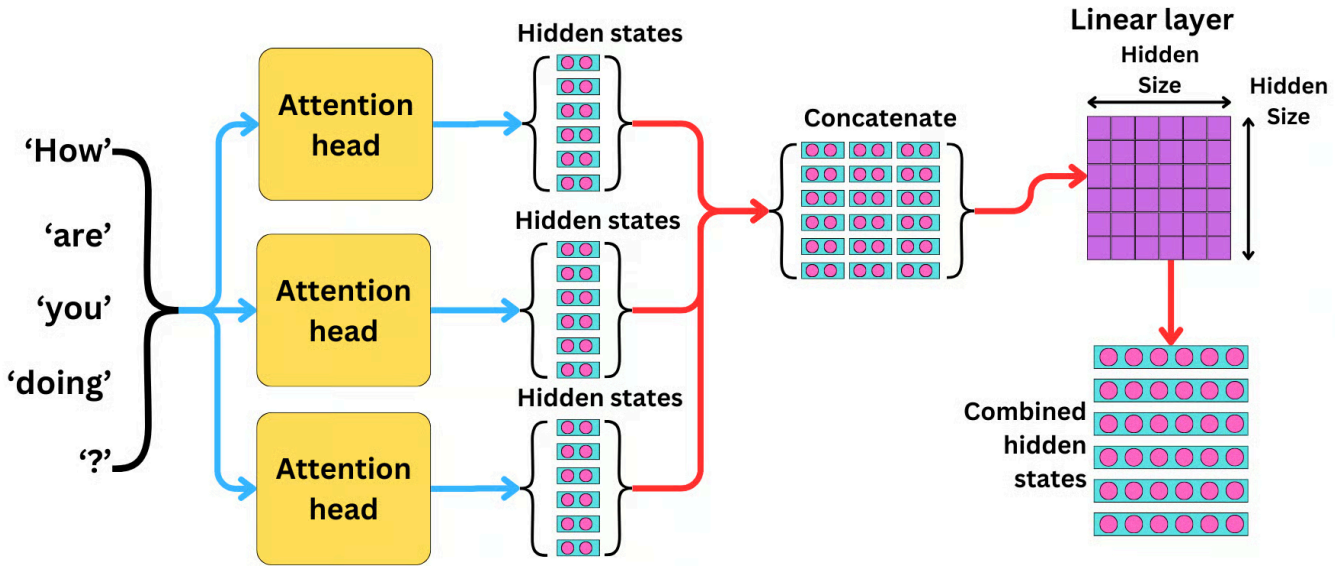
Визуализация матричных преобразований

```
Входная последовательность: ["Кот", "сидит", "на", "столе"]

Матрица оценок внимания (4×4):
            Кот    сидит    на    столе
  Кот     [ 0.7    0.1     0.1    0.1 ]
  сидит   [ 0.3    0.5     0.1    0.1 ]
  на      [ 0.1    0.2     0.3    0.4 ]
  столе   [ 0.1    0.2     0.3    0.4 ]
```

Строка «Кот» показывает: слово «кот» больше всего «смотрит» на само себя (0.7), а строка «столе» уделяет равное внимание предлогу «на» и самому «столу».

2.4 Multi-Head Attention: «Комитет экспертов»



Вместо одного механизма внимания трансформер использует **h параллельных «голов»** (heads). Каждая голова учится концентрироваться на разных аспектах текста:

- Голова 1 → синтаксические связи (подлежащее — сказуемое)
- Голова 2 → кореференции («он» = «Иван»)
- Голова 3 → семантическая близость
- Голова 4 → длинные зависимости через всё предложение

Небанальная аналогия: Multi-head attention — это как экспертный совет. Один эксперт читает текст на предмет логики, другой — на предмет эмоций, третий — грамматики. Потом их мнения объединяются в единый вывод.

Математически:

```
MultiHead(Q, K, V) = Concat(head₁, head₂, ..., headₕ) · W_O

где head_i = Attention(Q·W_Q_i, K·W_K_i, V·W_V_i)
```

В GPT-3 используется **96 голов внимания** с размерностью d_model = 12288. MachineLearningMastery.com[4]

2.5 Feed-Forward Network: «Нелинейный преобразователь»

После слоя внимания каждый токен проходит через **позиционно-независимую** двуслойную нейронную сеть:

```
FFN(x) = max(0, x·W₁ + b₁) · W₂ + b₂
```

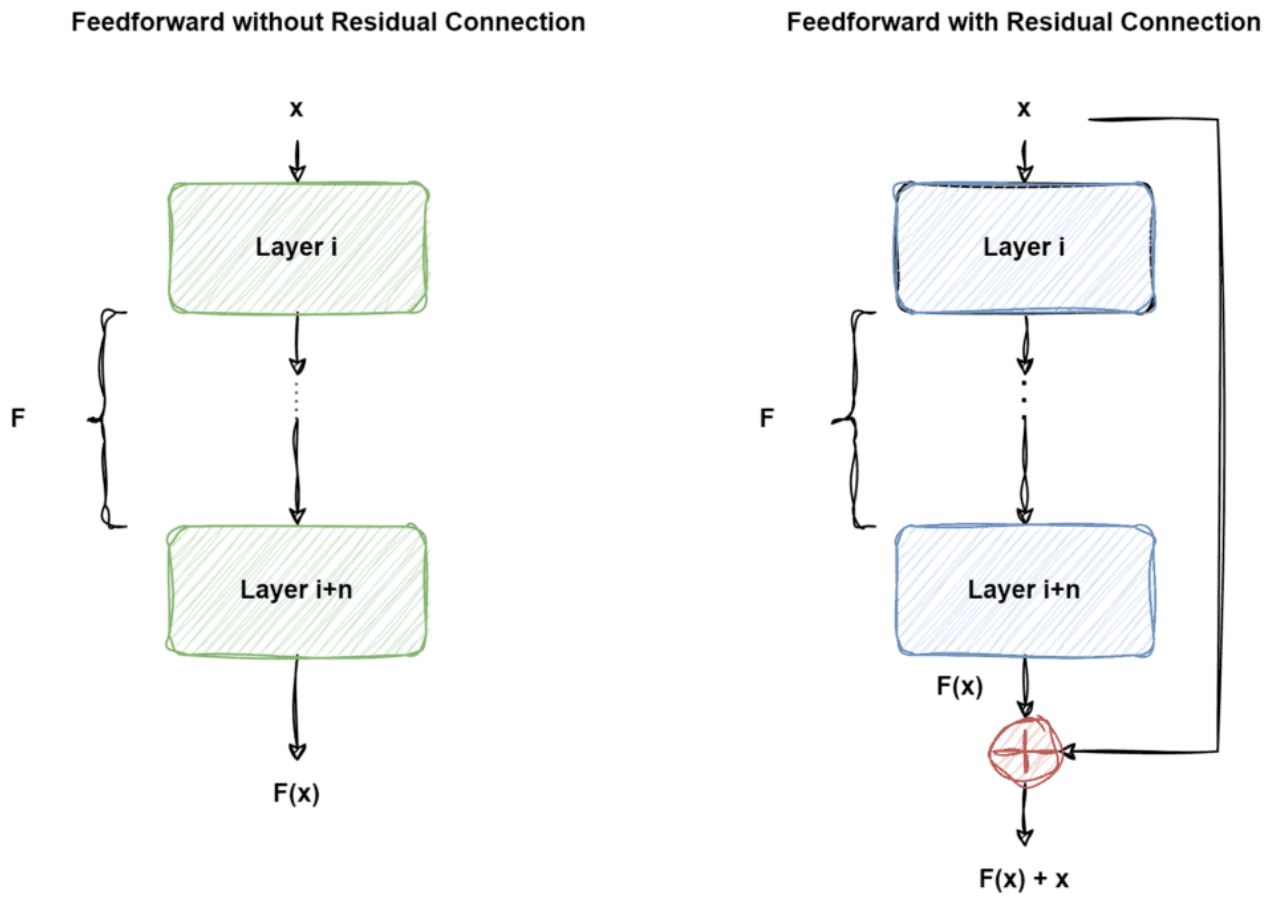
Архитектурная деталь:

- Входной размер: `d_model = 512`
- Скрытый слой расширяется в 4 раза: `d_ff = 2048`
- Обратно сужается до `d_model = 512`

Небанальная аналогия: Это как «химическая реакция». Вектор сначала «разворачивается» в широкое пространство (где много места для нелинейных взаимодействий), а потом «сворачивается» обратно, но уже обогащённый. medium.com[7]

Функция активации **ReLU** (или её улучшенная версия **GeLU** в GPT) вводит нелинейность — без неё все преобразования были бы линейными и модель не могла бы учить сложные паттерны.

2.6 Residual Connections: «Пожарные лестницы»



После каждого подблока (Attention и FFN) входной сигнал **добавляется** к выходному:

```
Output = LayerNorm(x + Sublayer(x))
```

Почему это критически важно? При обратном распространении ошибки через десятки слоёв градиент «затухает» (vanishing gradient problem). Остаточные связи создают **прямой путь** для градиентов — они могут «перепрыгнуть» через слои, сохраняя обучающий сигнал. Baeldung[8]

Небанальная аналогия: Пожарные лестницы в небоскрёбе. Если основной лифт сломан (градиент угасает), остаточные связи — это запасные выходы, по которым сигнал может беспрепятственно добраться до нижних этажей.

2.7 Layer Normalization: «Автоматический эквалайзер»

```
LayerNorm(x) = γ · (x − μ) / σ + β
```

где μ и σ — среднее и стандартное отклонение по **признакам** (не по батчу!).

Layer Normalization нормализует **каждый токен независимо** — это важно для последовательных данных, где длины текстов разные. Medium[9]

Свойство	Batch Normalization	Layer Normalization
Нормализация по	батчу (столбцы)	признакам (строки)
Зависит от размера батча	Да	Нет
Работает с padding	Плохо	Хорошо
Используется в трансформерах	Редко	Да, повсеместно

Часть III. Полный цикл обработки запроса

Реальный пример: запрос «Какова столица Франции?»

Шаг 0 — Токенизация

Текст разбивается на токены через алгоритм **BPE (Byte-Pair Encoding)**:

```
"Какова столица Франции?"
→ ["Как", "ова", "_сто", "лица", "_Фран", "ции", "?"]
```

Каждому токену присваивается числовой ID из словаря (vocab_size ≈ 50 000 для GPT).

Шаг 1 — Получение эмбеддингов

Каждый ID → вектор через матрицу эмбеддингов E (vocab_size × d_model):

```
"Как" (ID: 12847) → [0.23, -0.71, 0.45, ..., 0.12]  # вектор из 768 чисел
```

Шаг 2 — Добавление позиционного кодирования

```
X₀ = E("Как")    + PE(0)
X₁ = E("ова")    + PE(1)
X₂ = E("_сто")  + PE(2)
...
```

Шаг 3 — Прохождение через N блоков трансформера

Для каждого из 12–96 слоёв (в зависимости от размера модели):

```
┌──────────────────────────────────┐
│  БЛОК ТРАНСФОРМЕРА (повторяется N раз)  │
│                                  │
│  x → [Multi-Head Attention] → + x → LN │
│                                  │
│  x → [Feed-Forward Network] → + x → LN │
└──────────────────────────────────┘
```

Что происходит с нашим запросом:

В слоях внимания токен «Франции» начинает «смотреть» на токен «столица» — высокий вес внимания связывает эти два слова. К финальным слоям модель формирует **богатое контекстуальное представление**: «речь о географическом объекте — Франции, запрашивается её столица».

Шаг 4 — Проекция в словарь (Language Model Head)

Финальный вектор последнего токена умножается на транспонированную матрицу эмбеддингов:

```
Logits = H_final · Eᵀ   # вектор длиной vocab_size (~50 000)
```

Шаг 5 — Softmax и выборка

```
Probabilities = softmax(Logits / temperature)
```

Топ-5 вероятных следующих токенов:

```
"Париж"    : 0.847 ═══════════════════ ← выбирается
"Лион"     : 0.054 ═
"Марсель"  : 0.031
"столицей" : 0.021
"город"    : 0.011
```

Шаг 6 — Авторегрессивная генерация

Модель не генерирует весь ответ сразу — она **итеративно** добавляет по одному токену, каждый раз запуская полный forward pass:

```
Запрос:  "Какова столица Франции?"
→ Шаг 1: генерирует "Париж"
→ Шаг 2: "Париж" + контекст → генерирует "."
→ Ответ: "Париж."
```

Часть IV. Сводная диаграмма архитектуры

```
╔═══════════════════════════════════════════════╗
║             АРХИТЕКТУРА ТРАНСФОРМЕРА            ║
╚═══════════════════════════════════════════════╝
║                                               ║
║  ВХОД: "Какова столица Франции?"              ║
║             ↓                                 ║
║  [ТОКЕНИЗАТОР]  → ID токенов                   ║
║             ↓                                 ║
║  [EMBEDDING LAYER]  → векторы d_model          ║
║             ↓                                 ║
║  [POSITIONAL ENCODING]  → +синус/косинус       ║
║             ↓                                 ║
║  ┌──────────────────────────────────┐          ║
║  │  БЛОК ТРАНСФОРМЕРА (× N слоёв)   │          ║
║  │                                  │          ║
║  │  ┌──────────────────────────┐    │          ║
║  │  │  Multi-Head Self-Attention  │  │          ║
║  │  │  (Q·Kᵀ/√dk → softmax → ·V)  │  │          ║
║  │  └──────────────────────────┘    │          ║
║  │            ↓ + residual          │          ║
║  │       [Layer Norm]               │          ║
║  │            ↓                     │          ║
║  │  ┌──────────────────────────┐    │          ║
║  │  │  Feed-Forward Network       │  │          ║
```

```
│  │  │  (Linear → ReLU → Linear)    │  │          │
│  │  └──────────────────────────────┘  │          │
│  │            ↓ + residual            │          │
│  │       [Layer Norm]                 │          │
│  └────────────────────────────────────┘          │
│              ↓                                   │
│  [OUTPUT PROJECTION]  → logits (vocab_size)      │
│              ↓                                   │
│  [SOFTMAX]  → вероятности токенов                │
│              ↓                                   │
│  ВЫХОД: "Париж."                                 │
└──────────────────────────────────────────────────┘
```

Часть V. Ключевые инсайты и небанальные аналогии — сводка

Компонент	Небанальная аналогия
Параметры	Миллиарды регуляторов в органе — каждый настраивается обучением
Эмбеддинги	GPS-координаты смысла в многомерном пространстве
Позиционное кодирование	Часы с разной скоростью хода — уникальный «отпечаток» позиции
Self-Attention (QKV)	Поиск книги в библиотеке: запрос + ярлыки + содержимое
Softmax в attention	Голосование: сколько «внимания» уделить каждому токену
Multi-Head Attention	Экспертный совет, где каждый читает текст под своим углом
Feed-Forward	Химическая реакция: раскрыть, преобразовать, свернуть
Residual Connections	Пожарные лестницы для градиентов в глубоких сетях
Layer Normalization	Автоматический эквалайзер, убирающий «перегрузки» сигнала

Компонент	Небанальная аналогия
Авторегрессия	Написание письма по одной букве, читая всё написанное перед новым символом

Интерактивные ресурсы для углублённого изучения

- 🔬 Transformer Explainer (GPT-2 в браузере)10 — живая визуализация всех компонентов
- 📊 LLM Visualization (bbycroft.net)11 — трёхмерная интерактивная схема
- 📐 The Illustrated Transformer (Jay Alammar)12 — пошаговые иллюстрации
- 📖 Attention Is All You Need (arxiv)13 — оригинальная статья

Appendix: Supplementary Video Resources

🎥 Подробный обзор архитектуры Трансформер

🎥 Лекция 6: Трансформеры (часть 1). МФК «Нейронные сети ...

🎥 Лекция №10 "Трансформеры"